Markov TA Lab Current Project Guide

Date: 2026-05-17 (last refreshed after Phase D landed)

This guide explains how to use the project in its current state, what has been built, which commands to run, and how to interpret the generated research artifacts.

The project has progressed from a research prototype into a complete research lab with robustness checks, a macro Sharpe-lift gate, a paper-trading tracker, and a hatchling-built distribution with GitHub Actions CI. All five planned phases (A?E) are shipped. Open questions are now research questions, not engineering ones.

1. Project Goal

The Markov TA Lab is a Python research workspace for asking one central question:

```
When price interacts with support, resistance, or trend structure,
what is the probability of each next market state,
and does that probability create positive expected value after costs,
given the current macro and volatility regime?
```

The project is not a trading bot. It is not connected to a broker. It does not place trades. It is a research lab for market state modeling, transition analysis, expected value estimation, regime-aware backtesting, and *drift-monitored paper trading*.

The current research loop is:

```
download data (yfinance OR Financial Modeling Prep)
clean data
compute indicators
detect support/resistance zones
label market states
estimate transition probabilities
measure state expected value
run prototype + walk-forward backtests
compare against baselines (incl. random-label permutation)
condition results by volatility regime
cluster assets by structural behavior
pool sparse states by cluster
estimate Markov probability weighted EV
run parameter sensitivity grids + stability summary
attach bootstrap Sharpe CIs to reports
classify macro regimes and enforce Sharpe-lift gate
monitor state-distribution drift (KL divergence)
advance the stateful paper-trading tracker (no execution)
produce repeatable tables, reports, and a static HTML dashboard
```

2. Repository Location

The project is here:

```
C:\Users\CaveUser\Desktop\Project Project\markov-ta-lab
```

Start every terminal session from the repo root:

```
cd "C:\Users\CaveUser\Desktop\Project Project\markov-ta-lab"
```

The GitHub repo is:

```
https://github.com/geraldfillman/markov-ta-lab
```

3. Current Test Status

Run:

```
python -m pytest -q
```

Current expected result:

```
140 passed, 3 skipped
```

The three skipped tests are gated on optional libraries:

```
ruptures   (change-point detection)
hmmlearn   (HMM regimes)
hypothesis (property-based tests)
```

Install any of those and the corresponding test files start running. The core 140 tests cover:

```
project setup
CLI / script ergonomics
data cleaning and IO (yfinance + FMP)
FMP client + .env key handling
technical indicators
support/resistance levels
state labeling
Markov transition + forecast utilities
expected value metrics
state expectancy confidence intervals
sensitivity stability summary
block-bootstrap Sharpe CIs
report generation
readable + walk-forward backtests
random-label permutation baseline
signal-mask gating (used by macro + drift filters)
baseline comparisons
volatility regime classification
asset behavior clustering
cluster-pooled state expectancy
Markov probability weighted EV
walk-forward sensitivity grids
macro regime classification + Sharpe-lift gate
KL-divergence drift monitor
immutable PositionTracker (open/close/persist round-trip)
```

```
paper-trading orchestrator (gates + horizon-close behaviour)
module import smoke tests
```

4. Environment Notes

The project was designed for Python 3.11 and runs cleanly in a local .venv on this machine.

.venv/ is ignored by git and should not be committed.

PowerShell activation:

```
& ".\.venv\Scripts\Activate.ps1"
```

The & matters ? it tells PowerShell to run the script path.

If package installs fail on newer Python versions, it is usually because optional research packages (hmmlearn, ruptures, arch) need compatible wheels or Microsoft C++ Build Tools. The core 140 passing tests do not require those.

5. Experiment Universes

Two universes are defined in src/config.py:

```
FIRST_EXPERIMENT_SYMBOLS  (9 tickers, default everywhere)
  SPY QQQ IWM XLK SMH XLE XLF GLD TLT

DEFAULT_SYMBOLS  (15 tickers, opt-in via --full-universe)
  SPY QQQ IWM DIA XLK SMH XLE XLF XLI XLU XLV GLD SLV TLT UUP
```

Most scripts default to FIRST_EXPERIMENT_SYMBOLS. Add --full-universe to switch to the 15-ticker set.

6. One-Command Workflows

6.1 Core research pipeline

If processed data already exists, run the canonical research workflow in this order:

```
python scripts\build_state_expectancy_table.py
python scripts\run_narrow_backtest.py
python scripts\build_vol_conditioned_expectancy.py
python scripts\run_walkforward_backtest.py
python scripts\build_asset_clusters.py
python scripts\build_cluster_pooled_expectancy.py
python scripts\build_markov_weighted_ev.py
python scripts\run_sensitivity_tests.py
python scripts\run_sensitivity_stability.py
python scripts\run_sharpe_bootstrap_ci.py
python scripts\run_random_label_baseline.py
python scripts\build_dashboard.py
python -m pytest -q
```

If processed data does not yet exist:

```
python notebooks\01_data_download.py
```

6.2 Cross-provider research (FMP vs. yfinance)

```
python scripts\run_walkforward_backtest.py --provider yfinance
python scripts\run_walkforward_backtest.py --provider fmp
python scripts\run_provider_comparison.py --providers yfinance fmp
```

Produces reports/tables/provider_comparison.csv with <metric>__delta_<other>_minus_<base> columns.

6.3 Daily paper-trading job

```
python scripts\run_drift_monitor.py --provider fmp
python scripts\run_paper_trading_daily.py --provider fmp
```

Tracker lives at reports/paper_trading/tracker.json (atomic write). Decisions append to reports/tables/paper_trading_decisions.csv. Drift alerts gate new entries.

7. Data Pipeline

Main data script:

```
python notebooks\01_data_download.py
```

Optional FMP provider:

```
python notebooks\01_data_download.py --provider fmp
```

Pipeline steps:

1. Downloads OHLCV from yfinance (default) or FMP (--provider fmp).
2. Saves raw CSV files.
3. Adds technical indicators.
4. Adds support/resistance features.
5. Adds deterministic state labels.
6. Saves enriched parquet files.
7. Prints a missing-data report.

Outputs:

```
data/raw/*.csv
data/processed/*.parquet           (legacy path; also where load_processed falls back)
data/processed/<provider>/*.parquet (when you split snapshots per provider)
```

load_processed(symbol, provider="fmp") first checks data/processed/fmp/SYMBOL.parquet, then falls back to the legacy data/processed/SYMBOL.parquet.

Both raw and processed generated data files are ignored by git.

DEFAULT_END is currently 2026-05-16. yfinance treats the end date as exclusive, so the standard saved processed data includes the close through 2026-05-15.

8. Pipeline Flow

```
yfinance OR Financial Modeling Prep
  -> src.data.download_ohlcv  (provider-agnostic)
  -> src.indicators.add_indicators
  -> src.levels.detect_levels
  -> src.states.label_states
  -> data/processed/*.parquet

  -> src.metrics.state_expectancy_table
       -> reports/tables/state_expectancy.csv
  -> src.backtests.run_backtest_readable
       -> reports/tables/narrow_backtest_summary.csv
  -> src.backtests.run_walkforward_ev_backtest
       -> reports/tables/walkforward_backtest_summary[_<provider>].csv
  -> src.volatility.classify_vol_state
       -> reports/tables/vol_conditioned_state_expectancy.csv
  -> src.clustering.cluster_assets
       -> reports/tables/asset_behavior_clusters.csv
  -> src.metrics.cluster_pooled_state_expectancy_table
       -> reports/tables/cluster_pooled_state_expectancy.csv
  -> src.metrics.walkforward_markov_expected_value
       -> reports/tables/markov_weighted_ev.csv
  -> src.backtests.run_walkforward_sensitivity
       -> reports/tables/walkforward_sensitivity.csv
  -> src.metrics.sensitivity_stability_summary
       -> reports/tables/sensitivity_stability_summary.csv
  -> src.metrics.bootstrap_sharpe_ci_from_trades
       -> reports/tables/walkforward_sharpe_ci.csv
  -> src.backtests.baseline_random_label_walkforward
       -> reports/tables/random_label_baseline.csv
       -> reports/tables/baseline_comparison.csv (merged)
  -> src.macro.evaluate_macro_filter_sharpe_lift
       -> Sharpe-lift gate (>= 0.20 net-of-cost)
  -> src.drift.drift_alert
       -> reports/tables/drift_status.csv
  -> src.paper_trading.paper_trading_step
       -> reports/paper_trading/tracker.json
       -> reports/tables/paper_trading_decisions.csv
  -> src.dashboard.generate_dashboard
       -> reports/dashboard/index.html
```

9. Generated Outputs

Research tables in reports/tables/:

```
state_expectancy.csv
narrow_backtest_summary.csv
baseline_comparison.csv                        (merged with random_label rows)
vol_conditioned_state_expectancy.csv
```

```
walkforward_backtest_summary.csv
walkforward_backtest_summary_<provider>.csv   (per-provider variant)
walkforward_baseline_comparison.csv
walkforward_baseline_comparison_<provider>.csv
asset_behavior_clusters.csv
cluster_pooled_state_expectancy.csv
markov_weighted_ev.csv
walkforward_sensitivity.csv
sensitivity_stability_summary.csv             (Phase B)
walkforward_sharpe_ci.csv                     (Phase B)
random_label_baseline.csv                     (Phase B)
provider_comparison.csv                       (Phase B / D)
fmp_vs_yfinance_source_quality.csv
fmp_vs_yfinance_source_summary.csv
drift_status.csv                              (Phase D)
paper_trading_decisions.csv                   (Phase D)
```

Persisted paper-trading state:

```
reports/paper_trading/tracker.json            (atomic write, schema_version=1)
```

Human-readable experiment reports:

```
reports/runs/*.md
```

Static dashboard:

```
reports/dashboard/index.html
```

10. Module-by-Module Guide

10.1 ‘src/data.py‘

Provider-agnostic OHLCV download/clean/load. Key functions:

```
download_ohlcv(symbols, start, end, provider="yfinance")
load_processed(symbol, data_dir="data/processed", provider=None)
save_raw(data)
save_processed(data)
missing_data_report(data)
```

10.2 ‘src/fmp.py‘

FMP REST client + dotenv key loader.

```
load_fmp_api_key(dotenv_path=None)
normalize_fmp_ohlcv(records, symbol)
download_fmp_ohlcv(symbol, start, end)
FMPClient(api_key, transport=None)
```

The client masks the key in repr; missing-key errors never echo other secret values.

10.3 ‘src/indicators.py‘

Indicators describe the current bar using current or prior data only. Columns:

```
sma_20, sma_50, sma_200, ema_20, atr_14, rsi_14, bb_width_20,
return_1d, return_5d, return_10d, realized_vol_20, volume_zscore_20,
dist_to_sma_20, dist_to_sma_50, dist_to_sma_200
```

10.4 ‘src/levels.py‘

Support/resistance via prior rolling extremes (.shift(1) to prevent same-bar peek).

10.5 ‘src/states.py‘

12 deterministic state labels. Every bar gets exactly one state; warm-up bars become CHOP_OR_NO_EDGE.

10.6 ‘src/markov.py‘

Transition matrix + multi-step forecasts (matrix powers) + stationary distribution + walk-forward forecasts.

10.7 ‘src/metrics.py‘

Expected value tables + walk-forward EV + cluster-pooled + Markov-weighted EV. New in Phase B:

```
sensitivity_stability_summary(sensitivity, metric="sharpe")
bootstrap_sharpe_ci_from_trades(trade_returns, confidence=0.95, n_resamples=2000,
                                block_size=None, annualization=252.0,
                                avg_holding_period=1.0, random_state=0)
```

10.8 ‘src/backtests.py‘

Readable + walk-forward backtests with baselines. New in Phase B:

```
run_walkforward_ev_backtest(..., signal_mask: pd.Series | None = None)
baseline_random_label_walkforward(df, states, ..., seed=0)
compare_backtest_to_baselines(..., random_label_result=None)
```

10.9 ‘src/volatility.py‘

Volatility regime classifier (LOW_VOL / NORMAL_VOL / HIGH_VOL) + ATR-based position sizing.

10.10 ‘src/clustering.py‘

Asset-behavior clusters: realized volatility, trend persistence, reversal frequency, volume stability, average return.

10.11 ‘src/hmm_models.py‘

Real implementation. fit_hmm, label_regimes_by_behavior, regime_filter_signal, select_emissions.

10.12 ‘src/changepoints.py‘

Real implementation. detect_changepoints (Pelt via ruptures), changepoint_pause_signal, annotate_changepoints.

10.13 ‘src/macro.py‘

Macro regime filter with **Sharpe-lift acceptance gate**.

```
RISK_ON, RISK_OFF, NEUTRAL
MIN_REQUIRED_SHARPE_LIFT = 0.20

classify_macro_regime(spy_close, vix_close=None, spy_ma_window=200, ...)
conditional_transition_matrices(states, macro_regimes, n_states, alpha=1e-6)
macro_regime_distribution(macro_regimes)
macro_filter_signal(macro_regimes, allowed_regimes)
compare_conditional_to_unconditional(states, macro_regimes, n_states)
evaluate_macro_filter_sharpe_lift(df, states, macro_regimes, allowed_regimes, ...)
  -> {"sharpe_unfiltered", "sharpe_filtered", "sharpe_lift", "passes_gate", ...}
```

10.14 ‘src/drift.py‘ (Phase D)

KL-divergence drift monitor.

```
DEFAULT_KL_THRESHOLD = 0.10

state_frequency_distribution(states, n_states, alpha=1e-6)
kl_divergence(p, q, eps=1e-12)
drift_alert(training_states, current_states, n_states, threshold=DEFAULT_KL_THRESHOLD)
```

10.15 ‘src/positions.py‘ (Phase D)

Immutable paper-trading tracker.

```
@dataclass(frozen=True)
class Position:
    position_id: str; symbol: str
    signal_date: str; entry_date: str
    entry_price: float; horizon: int; state: int; signal_ev: float
    target_exit_date: str
    exit_date: str | None; exit_price: float | None
    net_return: float | None; closed_reason: str | None

@dataclass(frozen=True)
class PositionTracker:
    positions: tuple[Position, ...]
    # All mutations return new tracker instances.
    open(...)                           -> PositionTracker
    close(position_id, exit_date, ...)  -> PositionTracker
    open_positions(symbol=None)         -> tuple[Position, ...]
```

```
    has_open_position(symbol)           -> bool
    find(position_id)                   -> Position | None
    mark_to_market(prices)              -> dict[position_id, pnl]
    save(path)                          -> Path          # atomic
    @classmethod load(path)             -> PositionTracker
```

JSON schema is versioned (SCHEMA_VERSION = 1). Loading an unknown version raises.

10.16 ‘src/paper_trading.py‘ (Phase D)

Research only ? never places orders.

```
paper_trading_step(symbol, frame, tracker, today,
                   horizon=5, lookback=252, min_samples=10,
                   ev_threshold=0.0, cost_bps=5.0,
                   drift_blocked=False, macro_blocked=False)
  -> (updated_tracker, decisions)
```

Daily timing convention:

```
signal_date = yesterday (computed from data up to yesterday)
entry_date  = today     (entry at today’s close)
exit_date   = today + horizon business days
```

Step logic:

1. Close positions whose target_exit_date <= today.
2. Skip new entries if drift_blocked, macro_blocked, or there is already an open position.
3. Compute walk-forward EV for yesterday; open a new position at today’s close if EV > ev_threshold.

10.17 ‘src/dashboard.py‘ + ‘src/reports.py‘

Static HTML dashboard + Markdown experiment reports. The Research QA tab now includes:

```
Cluster-Pooled EV
Markov-Weighted EV
Sensitivity Tests
Parameter Stability (per symbol)     <- Phase B
Block-Bootstrap Sharpe CI            <- Phase B
```

The new cards render empty (not error) when sensitivity_stability_summary.csv or walkforward_sharpe_ci.csv haven’t been generated yet.

11. How To Use Each Feature

11.1 Download and enrich data

```
python notebooks\01_data_download.py
python notebooks\01_data_download.py --provider fmp
```

11.2 Build state expectancy table

```
python scripts\build_state_expectancy_table.py
```

11.3 Run full-sample narrow backtest (wiring only, lookahead-prone)

```
python scripts\run_narrow_backtest.py
```

11.4 Build volatility-conditioned expectancy

```
python scripts\build_vol_conditioned_expectancy.py
```

11.5 Run walk-forward backtest

```
python scripts\run_walkforward_backtest.py
python scripts\run_walkforward_backtest.py --provider fmp
python scripts\run_walkforward_backtest.py --provider fmp --full-universe
python scripts\run_walkforward_backtest.py --symbols SPY QQQ
```

11.6 Build asset behavior clusters

```
python scripts\build_asset_clusters.py
```

11.7 Build cluster-pooled state expectancy

```
python scripts\build_cluster_pooled_expectancy.py
```

11.8 Build Markov probability-weighted EV

```
python scripts\build_markov_weighted_ev.py
```

11.9 Run walk-forward sensitivity tests

```
python scripts\run_sensitivity_tests.py
```

11.10 Collapse the sensitivity grid into a stability summary (Phase B)

```
python scripts\run_sensitivity_stability.py
```

Output: reports/tables/sensitivity_stability_summary.csv.

11.11 Block-bootstrap Sharpe CIs (Phase B)

```
python scripts\run_sharpe_bootstrap_ci.py
python scripts\run_sharpe_bootstrap_ci.py --provider fmp --resamples 5000
```

Output: reports/tables/walkforward_sharpe_ci.csv. Block size = trade horizon to preserve serial structure between non-overlapping trades.

11.12 Random-label permutation baseline (Phase B)

```
python scripts\run_random_label_baseline.py
```

```
python scripts\run_random_label_baseline.py --provider fmp --seed 42
```

Writes reports/tables/random_label_baseline.csv and merges the rows into baseline_comparison.csv (dedupe by (symbol, model)).

If random_label performance is comparable to state_ev_strategy, the model is exploiting permutation artefacts.

11.13 Cross-provider comparison

```
python scripts\run_walkforward_backtest.py --provider yfinance
python scripts\run_walkforward_backtest.py --provider fmp
python scripts\run_provider_comparison.py --providers yfinance fmp
```

11.14 Drift monitor (Phase D)

```
python scripts\run_drift_monitor.py
python scripts\run_drift_monitor.py --provider fmp --recent 60 --threshold 0.10
python scripts\run_drift_monitor.py --full-universe
```

11.15 Daily paper-trading step (Phase D)

```
python scripts\run_paper_trading_daily.py
python scripts\run_paper_trading_daily.py --provider fmp
python scripts\run_paper_trading_daily.py --today 2026-05-16
```

Research only. This script never places orders.

11.16 Build static HTML dashboard

```
python scripts\build_dashboard.py
```

11.17 Run tests

```
python -m pytest -q
```

Expected: 140 passed, 3 skipped.

12. How To Inspect Results Quickly

Preview the walk-forward summary:

```
python -c "import pandas as pd;
print(pd.read_csv('reports/tables/walkforward_backtest_summary.csv'))"
```

Preview baseline comparison (now includes random_label rows):

```
python -c "import pandas as pd;
df=pd.read_csv('reports/tables/baseline_comparison.csv');
print(df[['symbol','model','total_return','excess_vs_buy_hold']].head(30))"
```

Preview stability summary:

```
python -c "import pandas as pd;
print(pd.read_csv('reports/tables/sensitivity_stability_summary.csv'))"
```

Preview Sharpe CIs:

```
python -c "import pandas as pd;
print(pd.read_csv('reports/tables/walkforward_sharpe_ci.csv'))"
```

Preview drift alerts:

```
python -c "import pandas as pd; df=pd.read_csv('reports/tables/drift_status.csv');
print(df[df['alert']==True])"
```

Inspect open paper positions:

```
python -c "from src.positions import PositionTracker;
t=PositionTracker.load('reports/paper_trading/tracker.json'); [print(p) for p in
t.open_positions()]"
```

13. Macro Acceptance Gate

The macro filter (src/macro.py) is held to playbook §3.12: it must improve walk-forward Sharpe by >= 0.20 net-of-cost or be dropped.

```
from src.macro import evaluate_macro_filter_sharpe_lift, RISK_ON

result = evaluate_macro_filter_sharpe_lift(
    df=frame,
    states=frame["state"],
    macro_regimes=macro_regimes,
    allowed_regimes=[RISK_ON],
    horizon=5, lookback=252, min_samples=10, cost_bps=5.0,
)
# result["sharpe_lift"], result["passes_gate"]
```

passes_gate is True iff sharpe_lift >= MIN_REQUIRED_SHARPE_LIFT (0.20).

14. Interpreting The Main Tables

14.1 ‘state_expectancy.csv‘

First-pass payoff inspection. Caveat: full-sample expectancy is useful for research, not trusted backtesting.

14.2 ‘walkforward_backtest_summary[_<provider>].csv‘

Primary prototype output. Columns: symbol, provider, horizon, lookback, min_samples, total_return, max_drawdown, sharpe, trade_count, win_rate, exposure_time, benchmark_total_return.

14.3 ‘baseline_comparison.csv‘

Strategy vs. baselines. Models surfaced: state_ev_strategy, buy_and_hold, ma_crossover, breakout, random_label. The excess_vs_buy_hold column is the main edge measure; the random_label row is the permutation null.

14.4 ‘walkforward_sensitivity.csv‘ + ‘sensitivity_stability_summary.csv‘

Sensitivity is the full grid; stability is the per-symbol collapse (median, std, IQR, share-of-negative-Sharpe). A robust result has tight IQR and low share-negative.

14.5 ‘walkforward_sharpe_ci.csv‘

A Sharpe whose CI brackets zero is not significant evidence of edge regardless of the point estimate.

14.6 ‘provider_comparison.csv‘

<metric>__delta_<other>_minus_<base> columns. A meaningful Sharpe delta means the conclusion is provider-sensitive; report which vendor was used.

14.7 ‘vol_conditioned_state_expectancy.csv‘

Same payoff question, conditioned on vol_state ? {LOW_VOL, NORMAL_VOL, HIGH_VOL}.

14.8 ‘asset_behavior_clusters.csv‘ + ‘cluster_pooled_state_expectancy.csv‘

Cluster-family payoffs are useful when a single-symbol state is too sparse.

14.9 ‘markov_weighted_ev.csv‘

Destination-state weighted EV. coverage and weighted_samples are diagnostic quality checks.

14.10 ‘drift_status.csv‘

alert == True means the recent window diverges from the training window past threshold (default 0.10 nats). Used as an entry gate by the paper-trading step.

14.11 ‘paper_trading_decisions.csv‘

Append-only log. One row per (run_date, symbol) decision, including the action (open, close, skip_entry) and a reason.

15. Current Prototype Results Snapshot

Latest walk-forward summary (per the last yfinance run before Phase B/D):

```
Best total return:           SMH
```

```
Strong risk-adjusted result: XLF
Weakest result:              XLE
TLT:                         approximately flat / slightly negative
```

Now that the random-label baseline, bootstrap Sharpe CIs, sensitivity stability summary, and macro gate exist, the next research question is whether those headline numbers survive **all four** robustness checks at once. The dashboard's Research QA tab is the easiest place to see that.

16. Important Bias and Risk Notes

Prototype hierarchy of trust:

```
run_narrow_backtest.py        -- wiring only (lookahead-prone)
run_walkforward_backtest.py   -- preferred prototype
+ random-label baseline       -- null-hypothesis check
+ block-bootstrap Sharpe CI   -- uncertainty band
+ sensitivity stability       -- parameter robustness
+ macro gate                  -- regime conditioning
+ provider comparison         -- vendor robustness
+ drift monitor               -- distribution-shift guardrail
```

Remaining limitations:

```
state definitions are still simple and deterministic
transaction costs are simplified (5 bps round-trip default; no liquidity-aware
slippage)
no train/test split around major stress periods (still walk-forward only)
clustering is interpretable but coarse (k = 3)
paper trading is daily-close fills, no microstructure
```

17. Current Git and Generated File Notes

Generated local files ignored by git include:

```
.venv/
.pytest_cache/
.pytest_tmp/
.test_output/
data/raw/*.csv
data/processed/**/*.parquet
reports/paper_trading/tracker.json   (regenerable from decisions log)
```

Research outputs in reports/tables/ and reports/runs/ are useful artifacts. Decide case by case whether to commit them. They document experiment results, but they can become stale when model logic changes.

18. Phase E ? Distribution (shipped)

The repo is now distributable:

- **PEP 621 / hatchling build** ? pyproject.toml declares the package, version (0.1.0),

runtime dependencies, optional extras (hmm, changepoint, garch, kalman, ta, backtest, viz, notebook, dev), and project URLs. python -m build --wheel produces markov_ta_lab-0.1.0-py3-none-any.whl. The wheel ships src/ as the importable package.
- **Editable install** ? pip install -e ".[dev]" is the recommended developer workflow. The conda path still works for notebook kernels.
- **Pinned dependencies** ? requirements.txt and pyproject.toml both use conservative lower-bounds (numpy>=1.26,<3.0, pandas>=2.2,<3.0, etc.). For full reproducibility generate a requirements.lock.txt via pip-compile.
- **CI** ? .github/workflows/test.yml runs the pytest suite on Python 3.11 and 3.12 for every push/PR and sanity-builds the wheel. .github/workflows/dashboard.yml rebuilds reports/dashboard/index.html from the committed reports/tables/*.csv and publishes to GitHub Pages on each push to main.

Enabling GitHub Pages

Once, in the repo settings: **Settings ? Pages ? Source: GitHub Actions**. The next push to main will publish the dashboard.

Optional research follow-ups (post-roadmap)

- Cluster-level transition matrices.
- Cluster-pooled walk-forward EV as a fallback when a single-symbol state is sparse.
- Volatility-conditioned walk-forward backtests.
- Liquidity-aware costs per symbol / per cluster.
- Train/test regime splits around major stress periods.
- A requirements.lock.txt generated via pip-compile for fully reproducible CI runs.